

RESEARCH PAPER

Smart Stock Analyzer: A DBMS-Driven Portfolio Management and Risk-Aware Recommendation System with Operating-System-Level Concurrency Control

Project Code: OSDBMS-V-2026-T139 | Team: Code Geeks

TEAM CREDITS

**Hariom Raturi**

Team Lead

Student ID: 24021325

Email: hariomraturi093@gmail.com

**Paarth Joshi**

Team Member

Student ID: 240212029

Email: paarthjoshi3120@gmail.com

**Suryadev Singh**

Team Member

Student ID: 240212376

Email: snnsurya31@gmail.com

Prepared as part of the Operating Systems and Database Management Systems (OS-DBMS) Project-Based Learning curriculum, B.Tech.

SPECIAL THANKS

We express our heartfelt gratitude to our mentor,



Shailendra Sir

for his invaluable help, patient guidance and constant encouragement throughout the development of the *Smart Stock Analyzer*. His insights into database design and operating-system concepts helped us turn a simple idea into a structured, well-reasoned system, and his feedback at every stage pushed us to think deeper and build better.

This work would not have been possible without his mentorship, and we remain sincerely thankful for the time and trust he invested in our team.

— *Hariom Raturi, Paarth Joshi & Suryadev Singh*
Team Code Geeks

Smart Stock Analyzer: A DBMS-Driven Portfolio Management and Risk-Aware Recommendation System with Operating-System-Level Concurrency Control

Hariom Raturi, Paarth Joshi, Suryadev Singh
Team Code Geeks — Project OSDBMS-V-2026-T139

Abstract

Retail investors and students who wish to learn about the stock market must usually collect price data, trade records and portfolio information from several disconnected sources, which makes it hard to see the overall state of their investments. This paper presents the design of the **Smart Stock Analyzer**, a unified application that lets a user register, browse stocks, record buy and sell transactions, maintain a portfolio, and obtain profit/loss, break-even and risk-aware recommendations from one place. A normalized relational database forms the core of the system and guarantees consistent storage of users, stocks, portfolios, transactions and logs. A rule-based decision engine converts a computed risk score into four graded responses, ranging from unrestricted access to a read-only mode. Operating-system concepts, namely processes, threads, synchronization and resource management, are applied so that concurrent trades and background price updates remain correct and efficient. The paper describes the motivation, architecture, database design, module design, OS integration and test strategy, and discusses limitations and future work. The system is intended as an educational demonstration of combined DBMS and OS principles and not as a source of financial advice.

Keywords: stock analysis, portfolio management, relational database, transaction processing, concurrency control, synchronization, risk scoring, recommendation system.

I. Introduction

Investment decisions depend on many interacting factors: current and historical prices, the risk profile of each stock, the composition of the investor's portfolio and the realised or unrealised return on past trades. Gathering and comparing this information is slow when it is spread across websites, spreadsheets and brokerage statements. Beginners in particular struggle to see the “big picture” because no single place shows what they bought, what it is worth now and how each trade changed their overall position.

The Smart Stock Analyzer addresses this by combining the essential functions of a stock-tracking tool into one lightweight system. Users can view stock information, record trades, track holdings, see profit and loss, find break-even points and receive elementary recommendations. All data are kept in a structured database so that storing, retrieving and updating information is reliable and efficient.

The project is also an exercise in connecting two core subjects. The **Database Management System** (DBMS) side covers schema design, relationships, integrity constraints and transactions. The **Operating System** (OS) side covers processes, threads, synchronization and resource management, which become important when several users trade at once while price updates run in the background.

The main contributions of this work are: (i) a modular architecture for a student-friendly stock analysis platform; (ii) a relational schema covering users, stocks, portfolios, transactions and logs; (iii) a risk-score-driven decision engine with graded responses; and (iv) a practical mapping of OS concurrency concepts onto trade processing.

II. Problem Statement and Motivation

Normal investment workflows have three recurring weaknesses. First, **fragmentation**: price data, trade history and portfolio value are found in different places, so users must switch between sources. Second, **complexity**: professional platforms offer powerful analytics but overwhelm students and first-time investors with features they do not need. Third, **lack of traceability**: without a central record, it is difficult to review how each trade affected the overall portfolio over time.

The problem addressed here can be stated as follows: *design a simple, reliable and database-backed system that lets an investor manage accounts, view stocks, record trades, monitor a portfolio and receive basic guidance, while remaining correct when many operations occur concurrently.* This is a realistic application that requires DBMS functionality such as user management, transaction recording and stock-availability tracking, and it naturally exercises OS concepts such as waiting, sleeping and sharing system resources.

III. State of the Art and Related Work

Modern stock-analysis platforms provide price tracking, portfolio management, transaction records, charting, profit/loss calculation, market news, screening tools and recommendations. They allow users to follow their money trail across holdings. However, many advanced platforms are difficult for beginners to learn, distribute information over different sections or applications, and depend on external services for analytics or AI-based suggestions.

From a systems perspective, trading platforms are classic examples of online transaction processing, where correctness under concurrency is essential. Silberschatz et al. [2] describe how atomicity, consistency, isolation and durability (the ACID properties) keep a database valid when many transactions execute together, while Silberschatz et al. [1] explain how synchronization primitives prevent race conditions between processes and threads. Zou and Su [3] analyse the design of a distributed multi-leg stock trading system and highlight the importance of reliable coordination between trading components. The Smart Stock Analyzer draws on these ideas but intentionally limits scope to a small, understandable feature set.

Aspect	Typical advanced platforms	Smart Stock Analyzer
Target user	Professional / active traders	Students and beginners
Feature set	Very large, complex	Small, essential, easy to learn
Data location	Spread across sections / apps	One unified database
Analytics	Often external or AI-dependent	Built-in, rule-based and transparent
Educational value	Low (black box)	High (DBMS + OS concepts visible)

Table I. Comparison with existing solutions.

IV. Objectives

The overall objective is an easy-to-use platform that combines the most important stock-investment functions in one place. The specific objectives are:

- Provide secure **registration and login** so that every investor has a private account.
- Allow users to **view shares** and their key details.
- Support **buy and sell transactions** with validation and a permanent transaction history.
- Maintain a **portfolio** showing holdings and tracking investments over time.
- Compute **profit/loss and break-even** values for individual holdings.
- Offer **basic recommendations** derived from stock-specific data.
- Store everything in a **structured relational database** with correctly related tables.
- Demonstrate **OS concepts** (processes, threads, synchronization, resource management) in a working system.

V. System Architecture

The system follows a layered, modular design, shown in Fig. 1. A user (investor or student) interacts through registration/login, stock and market-data views, buy/sell actions and portfolio/history access. Requests travel over HTTP/HTTPS to the **frontend** (web or desktop application), which provides the user-interface layer, input validation and result display. The frontend calls the **application server** through API calls.

The application server holds the backend logic: business logic, authentication and authorization, stock and portfolio services, transaction handling and the recommendation engine. It reads and writes the **database**

(DBMS), which contains the Users, Stocks, Portfolio, Transactions and Logs tables. Data from the application server and database also flow into a **data processing and analysis module** that performs stock analysis, risk assessment and recommendation generation. Its output feeds a **decision engine** that selects the response to present on the final **dashboard**, and every outcome is saved to the logs table.

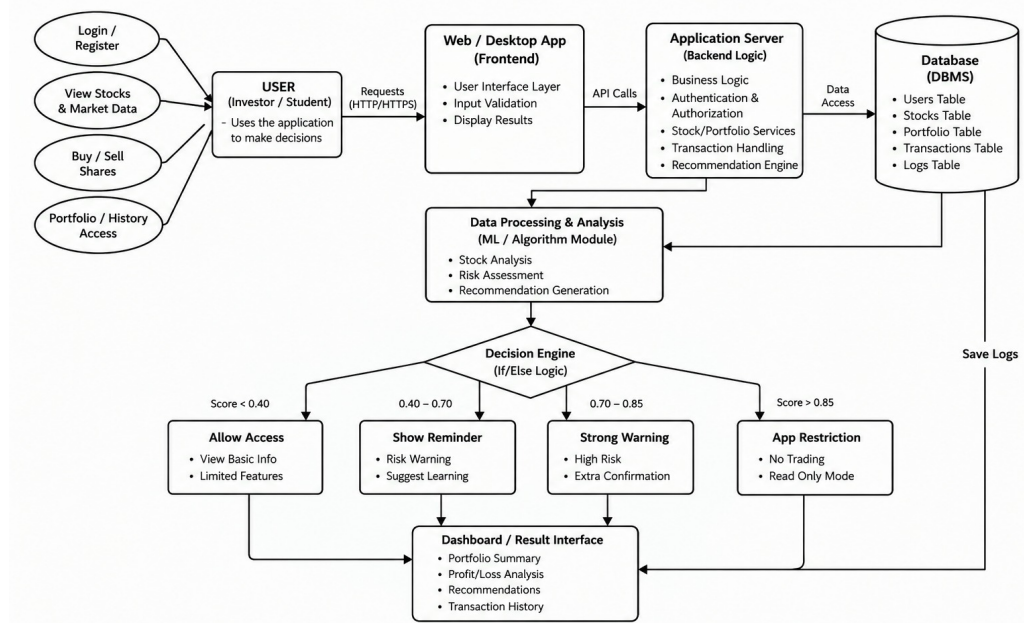


Fig. 1. High-level architecture of the Smart Stock Analyzer.

VI. Database Design

The database is the backbone of the system. Its design follows relational principles so that each fact is stored once, tables are linked by keys, and integrity rules prevent inconsistent data. Five principal tables are defined (Table II).

Table	Purpose	Key attributes (illustrative)
Users	Account and authentication data	user_id (PK), name, email (unique), password_hash, created_at
Stocks	Stock master data and availability	stock_id (PK), symbol (unique), company, current_price, available_qty
Portfolio	Current holdings of each user	portfolio_id (PK), user_id (FK), stock_id (FK), quantity, avg_buy_price
Transactions	Immutable record of every trade	txn_id (PK), user_id (FK), stock_id (FK), type (BUY/SELL), quantity, price, txn_time
Logs	Audit trail of actions and decisions	log_id (PK), user_id (FK), event, risk_score, created_at

Table II. Principal database tables.

Relationships. One user owns many portfolio rows and many transactions (one-to-many); one stock appears in many portfolio rows and transactions (one-to-many); the Portfolio table therefore resolves the many-to-many relationship between users and stocks. Foreign keys guarantee that no trade or holding can refer to a non-existent user or stock.

Integrity and transactions. Primary-key, unique, not-null and check constraints (for example, quantity > 0 and price > 0) enforce validity at the database level. A buy or sell is executed as a single atomic database transaction that (a) validates the request, (b) inserts the transaction record, (c) updates the portfolio holding, and (d) adjusts the stock's available quantity. Either all four steps commit or none do, which preserves the

ACID properties [2].

VII. Module Design

The application is divided into independently developed and tested modules, which keeps each part small and easy to verify.

A. User Interface and Account Module

This module covers registration, login, the view-shares screen and access to the portfolio. Input validation at this layer rejects empty or malformed fields before any request reaches the server.

B. Transaction Module

Users can buy and sell shares. A *validation* step checks that the user is authenticated, the stock exists, the requested quantity is positive, enough shares are available to buy, and the user holds enough shares to sell. Valid requests are committed atomically and stored as permanent transaction records, which can later be listed as the user's history.

C. Portfolio Module

The portfolio module tracks holdings and computes investment value. With q shares held at average purchase price p_{avg} and current market price p_{now} , the following measures are calculated:

Measure	Formula
Investment value	$V = q \times p_{now}$
Average cost (after a buy)	$p_{avg} = (q_{old} p_{avg,old} + q_{new} p_{buy}) / (q_{old} + q_{new})$
Profit / Loss	$PL = (p_{now} - p_{avg}) \times q$
Return (%)	$R = (p_{now} - p_{avg}) / p_{avg} \times 100$
Break-even price	$BEP = p_{avg} + (\text{total charges} / q)$

Table III. Portfolio calculations.

D. Recommendation Module

A simple recommendation component uses available stock data, such as recent price movement, to suggest whether a stock looks favourable, neutral or risky. It is deliberately elementary: its purpose is to demonstrate how analytics can be embedded in the application and not to predict the market.

VIII. Risk-Aware Decision Engine

After the data-processing module assesses a stock or an attempted action, it produces a **risk score** between 0 and 1. The decision engine applies if/else logic to this score and selects one of four responses (Table IV). This design makes the system's behaviour transparent and easy to explain and test, in contrast to opaque black-box models.

Risk score	Response	Behaviour presented to the user
< 0.40	Allow access	Basic information visible; limited features available
0.40 – 0.70	Show reminder	Risk warning displayed; learning material suggested
0.70 – 0.85	Strong warning	High-risk alert; extra confirmation required before proceeding
> 0.85	App restriction	Trading disabled; read-only mode

Table IV. Risk score thresholds and system responses.

All four paths lead to the dashboard, which shows the portfolio summary, profit/loss analysis, recommendations and transaction history. The chosen response and score are written to the Logs table so that every decision can be audited later. Because the thresholds are plain constants, they can be tuned without

changing the rest of the code.

IX. Operating-System Integration

Although the system is database-centred, a realistic trading application is inherently concurrent. Many users may submit orders at the same time while a background task refreshes stock prices. The project therefore applies four OS concepts [1].

Processes and threads. The application server handles each client request on its own thread (or worker process), so one slow request does not block others. A separate background thread periodically refreshes stock prices and recalculates portfolio values, sleeping between cycles so that it does not waste CPU time.

Synchronization. The stock's available quantity and a user's holdings are shared data. If two threads buy the last remaining shares simultaneously, a race condition could sell more shares than exist. To prevent this, the critical section that checks and updates quantities is protected using mutual exclusion (locks or database row-level locking), and database transactions are run at an isolation level that avoids lost updates. This also avoids deadlock by always acquiring locks in a fixed order (for example, by stock identifier).

Resource management. A bounded pool of database connections and worker threads limits memory and CPU usage. Requests that arrive when all workers are busy wait in a queue instead of crashing the system, illustrating waiting, scheduling and resource sharing in practice.

Mapping summary. The correspondence between OS concepts and system features is summarised below.

OS concept	Where it is used in the system
Threads / processes	One worker per client request; background price-update thread
Mutual exclusion	Protecting stock quantity and holdings during buy/sell
Waiting and sleeping	Price-refresh loop sleeps between updates; queued requests wait for a free worker
Deadlock avoidance	Fixed lock-ordering across stock and portfolio rows
Resource management	Connection pool and worker-pool limits

Table V. Mapping of OS concepts to system features.

X. Methodology, Technologies and Testing

The system is built using a **modular, incremental methodology**: requirements are gathered first, followed by architecture design, database schema design, module implementation and finally integration. Each module is developed and tested independently before being combined, which isolates defects early. The technology stack consists of a web or desktop frontend, an application layer exposing API endpoints, and a relational DBMS accessed through SQL.

Testing combines unit tests for individual functions (for example, profit/loss and average-cost calculations), integration tests for complete flows (register, login, buy, view portfolio, sell), validation tests with invalid input (negative quantities, unknown stock symbols, selling more than owned) and concurrency tests in which several simulated users trade the same stock at once to confirm that quantities never become negative or inconsistent. Error handling returns clear messages for invalid inputs rather than failing silently.

Test category	Example scenario	Expected outcome
Validation	Buy with quantity = 0 or negative	Request rejected with message
Business rule	Sell more shares than held	Request rejected; no data changed
Atomicity	Failure midway through a trade	Complete rollback; consistent state
Concurrency	Two users buy last available shares	Only one succeeds; stock never negative
Calculation	Verify P/L and break-even	Results match hand calculation

Test category	Example scenario	Expected outcome
Decision engine	Scores 0.30, 0.55, 0.80, 0.90	Correct response for each band

Table VI. Planned test cases.

XI. Expected Outcomes and Deliverables

The project is expected to deliver the following:

- **A functional trading engine** that monitors stocks, records buying and selling and supports analysis of the trade log.
- **Automated investment analytics** that determine profit/loss, calculate break-even and offer price-movement-based recommendations.
- **A practical OS and DBMS integration** showing relational data organization together with threading, process management and scheduling concepts working in a real application.
- **A validated, reliable system**, with tests across modules, input verification and error handling throughout the user flows.

XII. Assumptions and Limitations

The system assumes that users provide correct registration and transaction details, that stock prices and other analysis data are available and accurate, and that users have basic knowledge of shares. Limitations include the simplicity of the recommendation logic, dependence on the quality of the price data supplied, and the absence of real brokerage connectivity or real money handling. The Smart Stock Analyzer is a **learning tool**; it does not provide guaranteed financial advice and must not be used for real investment decisions.

Future work could include integration with live market data feeds, richer machine-learning-based risk models, charting and technical indicators, alerts and notifications, multi-factor authentication, and a distributed deployment in which the application and database run on separate machines, as discussed in [3].

XIII. Conclusion

This paper presented the design of the Smart Stock Analyzer, a modular, database-driven application that brings stock viewing, trading, portfolio tracking, profit/loss and break-even analysis and basic recommendations into a single, beginner-friendly system. A normalized relational schema with atomic transactions ensures data integrity, a transparent risk-score decision engine provides graded, auditable responses, and the application of threads, synchronization and resource management ensures correctness under concurrent use. The project demonstrates, in a realistic setting, how DBMS and OS principles complement each other, and provides a solid foundation for further extension.

References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [3] H. Zou and Z. Su, "Design and analysis of a distributed multi-leg stock trading system," in *Proc. IEEE Int. Conf. e-Business Eng. (ICEBE)*, Shanghai, China, 2010, pp. 112–119.